OpenMP Practical Lab Report

1 Calculating Pi

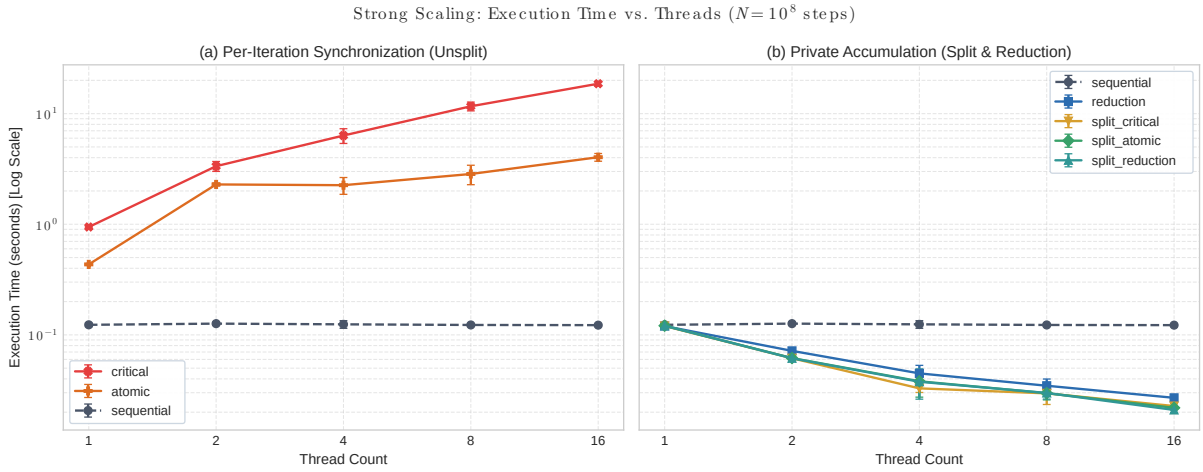


Figure 1: **Execution Time vs. Thread Count**

1.1 Performance analysis

I have displayed the differrent methods on the two graphs. Both graphs show the runtime of the benchmarks by thread count. The first grahp shows an increase of the runtime with the increase of the number of threads. It's because the synchronisation between threads happen in each loop, costing a lot of performance. In the other cases, where sequential and the manual splits are displayed, we can see a decline in execution time, which is because the calculation is actually distributed between the threads, and the costy synchronisation is executed only as many time as many threads there are (16 times in case of 16 threads for example).

2 Matrix-Vector Calculation

2.1 Sequential

2.1.1 Performance (Question 2.3)

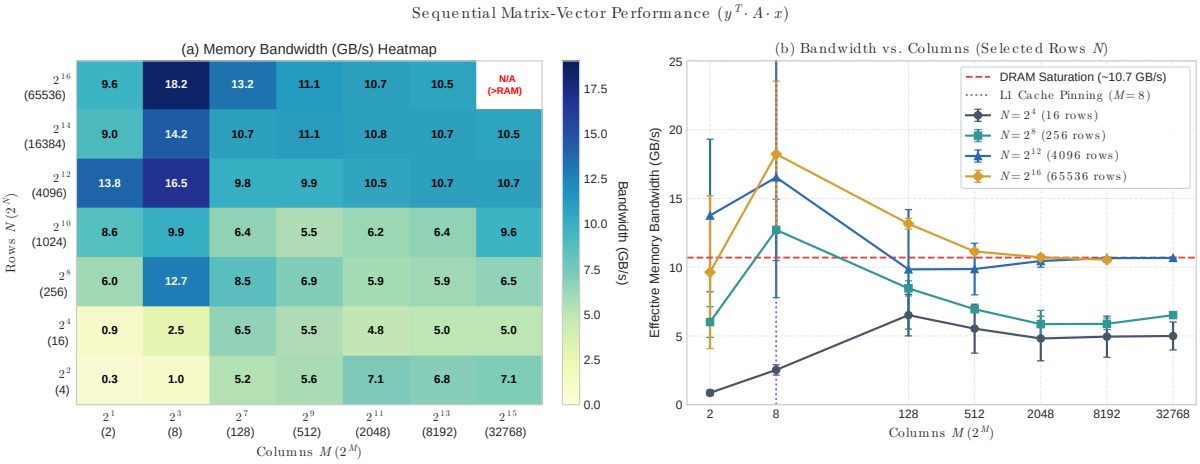


Figure 2: **Sequential Matrix-Vector Performance ($y^T \cdot A \cdot x$): (a) Bandwidth Heatmap**: Effective memory bandwidth (GB/s) across the full (N, M) parameter grid. The peak at $M = 8$ reflects optimal L1 cache retention for vector x (64 bytes = 1 cache line), while large matrices plateau at ≈ 10.7 GB/s due to main memory (DRAM) bandwidth saturation. The $\left(2^{\{16\}}, 2^{\{15\}}\right)$ cell is omitted due to 32-bit integer overflow and RAM limits. **(b) Bandwidth vs. Columns**: Clean scaling curves for representative row sizes ($N \in \left\{2^4, 2^8, 2^{\{12\}}, 2^{\{16\}}\right\}$), highlighting the $M = 8$ cache spike and convergence to the ≈ 10.7 GB/s DRAM ceiling.

2.1.2 Analysis (Question 2.4)